ADITYA UNIVERSITY

School of Computing

Department of Computer Science and Engineering

NATURAL LANGUAGE PROCESSING

A Cornerstone Project Report on

"CampusAI Academic Question Answering System"

Intent Recognition and Evidence Based Curriculum Retrieval

Submitted by

K Mohan Vamsi Reddy	**(24B11CS197)**
K Harsha Vardhan	**(24B11CS237)**
Y Pavanputra	**(24B11CS504)**
V Jaya Sai Krishna	**(24B11CS514)**

in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering

Under the guidance of

Dr.T Sudha Rani

Assistant Professor, Department of CSE

ADITYA UNIVERSITY, Surampalem

Academic Year 2026-2027

ADITYA UNIVERSITY

School of Computing

Department of Computer Science and Engineering

NATURAL LANGUAGE PROCESSING

CERTIFICATE

This is to certify that the Cornerstone project work entitled **"CampusAI Academic Question Answering System"** is submitted by

K Mohan Vamsi Reddy	**(24B11CS197)**
K Harsha Vardhan	**(24B11CS237)**
Y Pavanputra	**(24B11CS504)**
V Jaya Sai Krishna	**(24B11CS514)**

in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering during the academic year 2026-2027.

Project Guide
Dr.T Sudha Rani
Assistant Professor,
Department of CSE

Head of the Department
Dr. A. Phani Sridhar
Associate Professor & HOD
Department of CSE

DECLARATION

We hereby declare that the project entitled **"CampusAI Academic Question Answering System"** is our original work and is submitted to **ADITYA UNIVERSITY, Surampalem**, in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree or diploma in this or any other institution. We also confirm that this work has been carried out with academic honesty and complies with the Institutional Academic Integrity and **AI Usage Policy.**

Place: Surampalem
Date: _____

By

K Mohan Vamsi Reddy	**(24B11CS197)**
K Harsha Vardhan	**(24B11CS237)**
Y Pavanputra	**(24B11CS504)**
V Jaya Sai Krishna	**(24B11CS514)**

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Dr.T Sudha Rani, Assistant Professor**, who has guided us throughout the project. Their valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Hari J, Professor, Dr. Siva Satya Prasad P, Assistant Professor, Dr. Chandra Sekhar K, Associate Professor**, Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, **Dr. A. Phani Sridhar, Associate Professor & HOD**, for their inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing**, for his encouragement and support during our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar; Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences); Dr. S. Rama Sree, Pro Vice-Chancellor (Academics); Dr. M.B. Srinivas, Vice-Chancellor; and Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

ABSTRACT

CampusAI is an academic question-answering system designed to help students locate course, syllabus and curriculum information for Aditya University. The project combines natural-language query understanding with retrieval from eight B.Tech curriculum PDFs covering 6,502 pages. A saved retrieval index contains 1,039 structured passages, while the intent dataset contains 1,201 labelled queries across 30 categories in English, Telugu and code-mixed language.

The system preserves course codes, department names, semester numbers and unit references during preprocessing. Its trained intent model combines word and character TF-IDF features with a weighted soft-voting ensemble of calibrated LinearSVC, Logistic Regression and Complement Naive Bayes. Exact academic questions can use structured SQL retrieval; broader questions use lexical BM25 and dense semantic retrieval with reciprocal rank fusion. The application includes answer generation, source citations, ambiguity handling and abstention for unsupported real-time schedules.

The implementation uses a FastAPI backend, SQLAlchemy data models and a React and TypeScript interface with chat, course exploration, semester browsing and comparison views. The saved classifier evaluation reports 96.27% top-1 accuracy, 99.17% top-3 accuracy and a macro F1 score of 0.9196 on 241 test queries. A separate 16-query system benchmark is analysed with its limited fact checks and heuristic grounding labels. Hard-coded retrieval scores are excluded from claims of measured retrieval quality.

The project demonstrates how a domain-specific NLP application can connect student questions to inspectable institutional evidence. Its evaluation also identifies the need for stronger extraction checks, more balanced intent data, independently labelled retrieval tests and route-specific end-to-end validation.

Keywords: CampusAI, Natural Language Processing, Intent Classification, TF-IDF, Hybrid Retrieval, Retrieval Augmented Generation, Academic Helpdesk, Source Attribution.

TABLE OF CONTENTS

Title Page	i
Certificate	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi-vii
List of Figures	viii
List of Tables	ix
Chapter 1 Introduction	1
1.1 Introduction to the Project	1
1.2 Problem Statement	1
1.3 Objectives of the Project	2
1.4 Scope of the Project	2
Chapter 2 System Analysis and Design	3
2.1 Proposed System	3
2.2 System Architecture and System Workflow	3
2.3 Module Description	4
2.4 Hardware Requirements and Software Requirements	5
2.5 Tools and Technologies	6
Chapter 3 Dataset and Text Preprocessing	7
3.1 Dataset Description and Data Format	7
3.2 Data Collection	8
3.3 Data Preprocessing	9
Chapter 4 Question Understanding and Evidence Recognition	10
4.1 Need for Question and Evidence Recognition	10
4.2 Topic and Relationship Recognition	10
4.3 Question Classification	11
4.4 Evidence Mapping and Worked Example	11
Chapter 5 Answer Generation Validation and Execution	12
5.1 Retrieval and Reading-Comprehension Approach	12
5.2 Answer Generation	12
5.3 Validation and AI Response Handling	13
5.4 Request Execution and Bounded Failover	13
5.5 Handling of Different Outcomes	13

TABLE OF CONTENTS continued

Chapter 6 Implementation 14
6.1 User Question Input 14
6.2 Preprocessing Implementation 14
6.3 Question Understanding Implementation 15
6.4 AI Explanation Implementation 15
6.5 Important Code Paths 15
Chapter 7 Result Visualization and Dashboard 16
7.1 Dashboard Design 16
7.2 Dashboard Statistics 16
7.3 Result Presentation Types 16
7.4 Charts and Graphs 17
7.5 Sample Questions and Expected Outcomes 17
7.6 Demonstration and Interface States 19
7.7 Results and Discussion 21
Chapter 8 Conclusion and Future Enhancements 22
8.1 Conclusion 22
8.2 Limitations 22
8.3 Future Enhancements 22
References 23
Appendix A Source Code for Intent Features 24
Appendix B Source Code for Preprocessing 25
Appendix C Source Code for Query Handling 26
Appendix D Intent Dataset Distribution 27
Appendix E Reproducibility and Result Provenance 28
Appendix F Requirements and Run Instructions 29
Appendix G Detailed NLP and ML Code 30

LIST OF FIGURES

2.1 Layered architecture of CampusAI 3

2.2 Document ingestion and question-answering workflow 4

3.1 Evidence preservation and query feature preparation 9

7.1 Saved classifier test metrics on 241 queries 17

7.2 Saved confusion matrix for the 30-intent classifier 18

7.3 Assistant interface in a saved UI verification capture 19

7.4 Course catalogue in a saved UI verification capture 19

7.5 Semester view in a saved UI verification capture 20

7.6 Comparison view in a saved UI verification capture 20

G.1 NLP and ML library integration pipeline 30

LIST OF TABLES

2.1 Modules and responsibilities 4
2.2 Recommended demonstration hardware 5
2.3 Software requirements 5
2.4 Tools and technologies used 6
3.1 Curriculum PDF corpus verified from local files 7
3.2 Query language distribution and index composition 8
3.3 Illustrative processing of an academic question 9
4.1 Question-to-evidence relationships 10
4.2 Intent ensemble configuration 11
4.3 Worked example checked against the original PDF 11
5.1 Answer pathways and outputs 12
5.2 Response outcomes and interpretation 13
6.1 Principal API routes inspected in the project 14
6.2 Implementation files and responsibilities 15
7.1 Interface views and result presentation 16
7.2 Demonstration questions and intended outcomes 17
7.3 Selected lower-performing intent classes 18
7.4 Recorded evaluations and their limits 21
E.1 Reproducibility and validation checks 28
G.1 Python libraries and their integration roles 32
G.2 Supporting NLP and ML dependencies 33
G.3 Data flow through the NLP and ML components 48

CHAPTER 1
INTRODUCTION

1.1 Introduction to the Project

University curriculum handbooks contain course codes, credits, contact hours, semester structures, unit topics and reference books. Students often know what they want to ask but do not know the correct document, page or academic terminology. Searching separate department handbooks creates repeated effort, especially when the same course appears in several programmes.

CampusAI provides a conversational interface over Aditya University curriculum material. A student can ask about a course code, request a semester subject list, inspect a unit syllabus or compare departments. The application extracts the relevant academic entities and uses the requested information type to select an appropriate retrieval path.

The project combines supervised intent recognition, rule-based entity extraction, structured database queries and retrieval augmented generation. The aim is to return a useful answer together with information that allows the student to inspect its source. The underlying curriculum remains the authority for academic decisions.

1.2 Problem Statement

A general language model may produce a plausible academic answer without consulting the correct curriculum version. Keyword search alone can also retrieve unrelated material when abbreviations are ambiguous or when course titles overlap. Numerical fields are particularly vulnerable because PDF tables may place credits and total contact hours close together.

The problem addressed is therefore to identify the student's intent, preserve the exact course and department context, retrieve relevant institutional evidence and communicate uncertainty when support is incomplete. A successful interface must make source inspection practical and distinguish a model confidence score from independently verified answer correctness.

1.3 Objectives of the Project

Academic access: organise official curriculum documents so that students can find semester subjects, course details, syllabus units, outcomes and reference material through ordinary questions.

Query understanding: classify questions into 30 intent categories and extract entities such as course codes, course names, departments, semesters and unit numbers. Preserve abbreviations and support the language forms represented in the dataset.

Evidence retrieval: route exact facts through structured queries where available and use complementary lexical and semantic search for descriptive material. Retain document identifiers and page references throughout processing.

Responsible responses: request clarification for ambiguous expressions and avoid inventing live examination schedules or personal student information. Present source citations and evidence status alongside answers.

Evaluation: assess the intent model using stratified splits, cross-validation, class-specific precision and recall, and a confusion matrix. Keep classifier performance separate from retrieval relevance and end-to-end answer quality.

1.4 Scope of the Project

The current corpus covers eight B.Tech engineering departments under Curriculum 2024, Version 3.0. The interface includes an academic assistant, course catalogue, semester view, department comparison and administrative views. The implementation supports structured academic records, local retrieval indices and configurable generation adapters.

The scope does not establish access to private student grades, individual attendance records or authenticated live timetables. The training dataset is a project-curated query collection, not a representative survey of all students. Multilingual examples and language handling are present, but separate multilingual retrieval and answer-quality scores have not been established.

CHAPTER 2
SYSTEM ANALYSIS AND DESIGN
REQUIREMENT ANALYSIS

2.1 Proposed System

CampusAI separates the browser interface, API orchestration, language understanding, academic retrieval and answer presentation. Curriculum ingestion creates a reusable knowledge base; query-time processing selects evidence from that knowledge base. This avoids repeatedly scanning entire PDFs for each question.

2.2 System Architecture and System Workflow

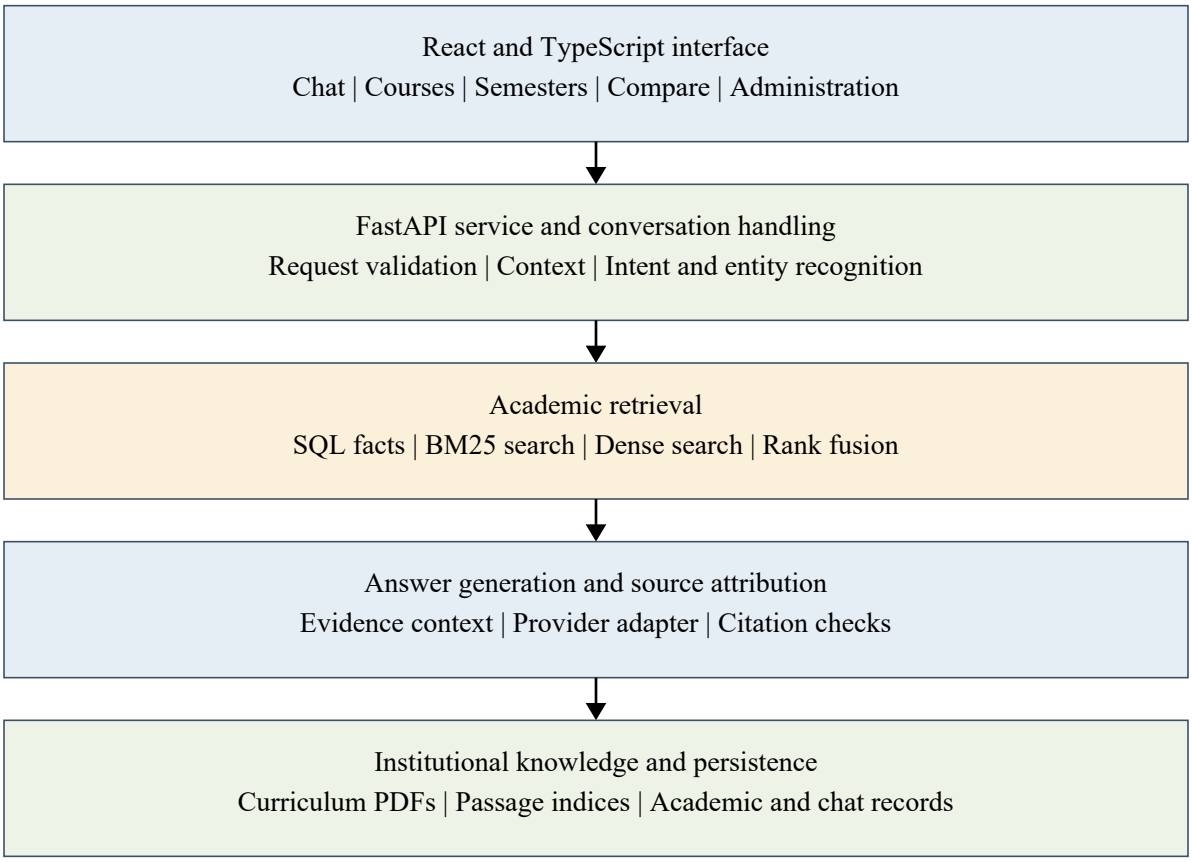


Figure 2.1: Layered architecture of CampusAI

The backend exposes both a versioned conversational API used by the frontend and additional core NLP endpoints. They share components but have different orchestration paths; results from one route should not automatically be attributed to the other. [P5]

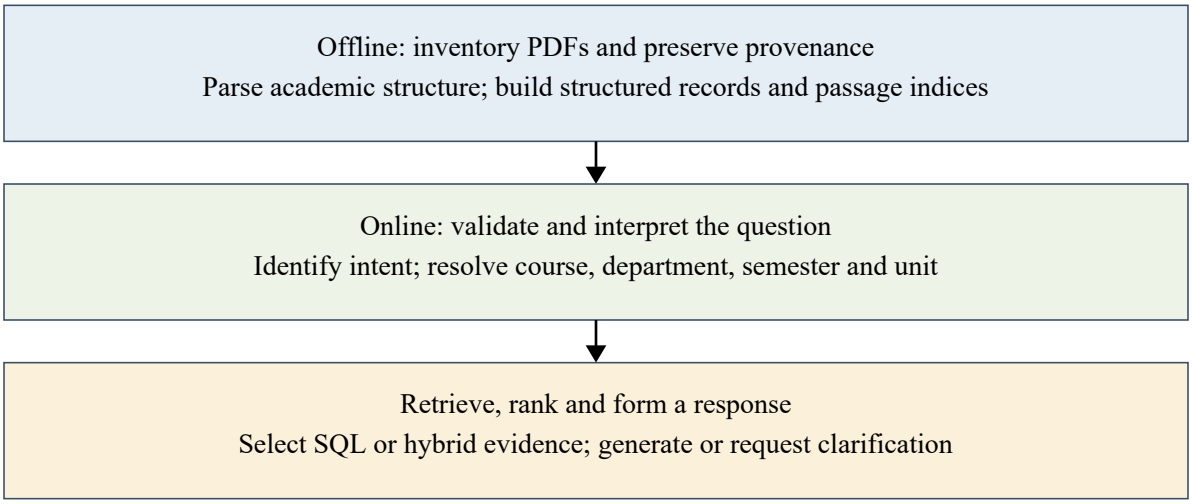


Figure 2.2: Document ingestion and question-answering workflow

2.3 Module Description

Module	Responsibility
Document ingestion	Inventory files, extract tables and syllabus structure, retain source pages.
Query understanding	Normalise text, predict intent, extract entities and identify ambiguity.
Academic storage	Model departments, courses, offerings, units, books and provenance.
Retrieval	Use structured queries, BM25, dense vectors and rank fusion.
Response generation	Prepare evidence context, call an adapter and return answer metadata.
Interface and feedback	Expose chat, catalogue, semester, comparison and source views.

Table 2.1: Modules and responsibilities

The multi-intent component decomposes compound questions into smaller tasks, such as listing subjects and retrieving their prerequisites. Each task needs its own evidence; successful decomposition alone does not demonstrate that every requested fact has been answered.

2.4 Hardware Requirements and Software Requirements

The following hardware profile is a practical recommendation for a local academic demonstration. It is not a measured minimum requirement or a throughput guarantee. Storage and memory needs depend on downloaded embedding models, optional rerankers, the database and the generation provider.

Component	Recommended configuration
Processor	Modern 64-bit multicore CPU
Memory	8 GB minimum planning allocation; 16 GB preferred
Storage	Several GB free for Python packages, model caches and documents
GPU	Optional; CPU execution is supported by the configuration
Connectivity	Required for initial model downloads and remote generation providers

Table 2.2: Recommended demonstration hardware

Component	Use in the project
Python 3.11 or later	Backend, NLP training, ingestion and evaluation
Node.js and npm	Frontend dependency installation and Vite development server
PostgreSQL or SQLite	SQLAlchemy-backed academic and conversational persistence
Web browser	Responsive React interface and PDF source viewing
Optional provider credentials	Only when selecting a remote answer-generation adapter

Table 2.3: Software requirements

The backend dependency file specifies minimum versions for FastAPI, SQLAlchemy, scikit-learn, Sentence Transformers, PyTorch, PDF extraction packages and other libraries. The training script additionally imports packages such as Matplotlib, Seaborn and NLTK; a reproducible environment must include these explicit requirements.

2.5 Tools and Technologies

Technology	Project role
Python and FastAPI	Typed request handling and backend orchestration
React, TypeScript and Vite	Interactive browser application
SQLAlchemy	Relational models and academic query access
PDF extraction libraries	Read curriculum text and structured table content
scikit-learn	TF-IDF, calibrated classification, ensemble training and metrics
Sentence Transformers	Dense sentence embeddings and optional cross-encoder scoring
BM25 and NumPy	Lexical ranking and vector similarity computation
JSON, CSV and pickle/joblib	Manifests, query dataset, indices and model artefacts

Table 2.4: Tools and technologies used

Database Design

The model source defines 20 tables. Academic entities include institutions, departments, programmes, curriculum versions, courses, semesters, course offerings, course units, course outcomes, textbooks, minor streams and minor-stream courses. Provenance is represented through source documents and extraction issues.

Users, chat sessions, chat messages, response citations, feedback and audit logs support application interaction. A course offering connects a course to its programme context and semester; units and outcomes are separate records rather than columns repeated inside a course row. This structure supports filtered queries and keeps source references available for display.

The local SQLite file inspected for this report contains no tables. Consequently, this report does not present README database row counts as a verified current database snapshot. The saved passage index is assessed separately in Chapter 3.

CHAPTER 3
DATASET AND TEXT PREPROCESSING

3.1 Dataset Description and Data Format

CampusAI uses two distinct datasets: a curriculum evidence corpus and a labelled query corpus. The evidence corpus supplies answers; the query corpus trains the model to recognise the information requested. Combining their counts would obscure their different purposes.

Department	Code	Pages
Civil Engineering	CE	728
Computer Science and Engineering - Data Science	CSE-DS	833
Computer Science and Engineering	CSE	808
Electrical and Electronics Engineering	EEE	808
Electronics and Communication Engineering	ECE	806
Information Technology	IT	814
Mining Engineering	MIN	749
Petroleum Technology	PT	956
Total	8 PDFs	6,502

Table 3.1: Curriculum PDF corpus verified from local files

The source manifest identifies Curriculum 2024, Version 3.0 and records file checksums. Individual page counts range from 728 to 956. Source page counts were checked against the eight local PDF files. Page availability does not imply that every table or course has been extracted correctly.

The saved dense array has shape **1,039 x 384**. Its row count agrees with the 1,039 passages in the metadata index. The configured all-MiniLM-L6-v2 model produces 384-dimensional vectors. [4, P1]

3.2 Data Collection

The query CSV is assembled from project intent examples and the structured intent catalogue. Its fields are query_id, query, intent, domain, language, course, department, semester and unit. The inspected file contains 1,201 rows, 30 intent labels and no exact duplicate query strings. The generation and export workflow does not establish that these are independently collected student conversations. [P2]

Dataset aspect	Category	Count
Query language	English (en)	850
Query language	Telugu (te)	48
Query language	Telugu-English (te-en)	303
Index passage type	Unit syllabus	603
Index passage type	Course overview	245
Index passage type	Textbooks	127
Index passage type	Semester summary	64

Table 3.2: Query language distribution and index composition

The two largest intents are unit_syllabus (229 rows) and semester_subjects (197 rows). Together they account for 426 of 1,201 examples, or 35.47%. The smallest class, institutional_leadership, contains 12 rows. This imbalance makes macro F1 and per-class analysis necessary alongside overall accuracy.

Passages carry a chunk identifier, passage type, course code and title, department, semester number, unit number, source document identifier, source page and additional metadata. This schema supports filtered retrieval and source links. However, counts of passage types are not interchangeable with counts of unique database courses or complete syllabus units.

3.3 Data Preprocessing

The classifier preprocessor lowercases text, normalises quotation marks and spacing, and tokenises alphanumeric and Telugu-script sequences. Course codes and protected academic terms bypass lemmatisation. Stopword removal is disabled in the default preprocessor, so words that help identify a question remain available to the model.

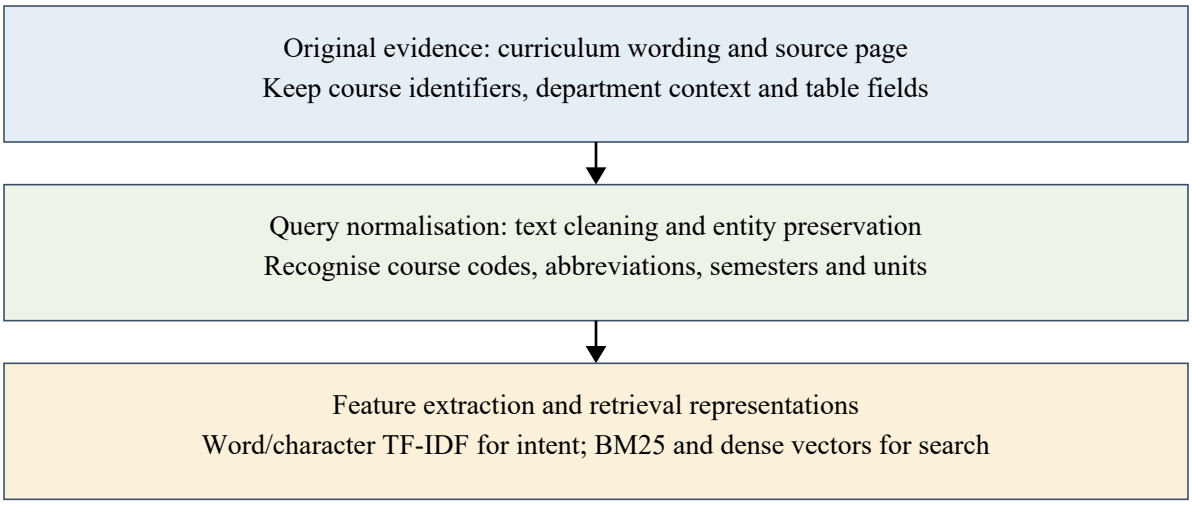


Figure 3.1: Evidence preservation and query feature preparation

Stage	Example
Input	How many credits does Operating Systems carry?
Protected information	Course title: Operating Systems; requested field: credits
Expected intent	course_credits
Evidence target	Operating Systems, code 241CS013, CSE PDF page 28

Table 3.3: Illustrative processing of an academic question

Data quality check: the CSE PDF on page 28 lists Semester IV total credits as 20 and total hours as 32. The saved semester-summary passage records total credits as 32. This concrete mismatch shows why numerical extraction must be checked against the original table before an answer is treated as authoritative.

CHAPTER 4
QUESTION UNDERSTANDING AND EVIDENCE RECOGNITION

4.1 Need for Question and Evidence Recognition

The same course can support several questions with different answer requirements. A request for credits should return a numerical field; a unit-syllabus question needs the correct unit text; a comparison requires evidence from more than one branch or course. Intent classification provides the routing signal, while entities identify the academic record.

4.2 Topic and Relationship Recognition

Requested relationship	Entities to resolve	Evidence needed
Course to code or credits	Course title, department	Course record or official curriculum row
Semester to subjects	Department, semester	Semester offerings and source table
Course to unit topics	Course title/code, unit	Matching unit-syllabus passage
Course to prerequisites	Course and curriculum context	Explicit prerequisite entry
Department comparison	Both departments, semester	Two compatible semester structures

Table 4.1: Question-to-evidence relationships

Abbreviations need context. For example, DS may refer to Data Structures or to the Data Science programme. The entity extractor can mark the query as ambiguous and return alternatives for clarification. Semester and unit references are also normalised so that forms such as III and 3 can be matched consistently.

The project includes conversation context and query rewriting components for follow-up questions. Their presence supports a conversational design, but a reliable multi-turn evaluation must still test whether the correct course and department remain attached to later questions.

4.3 Question Classification

The trained model combines word TF-IDF n-grams of lengths 1 to 3 with character n-grams of lengths 2 to 5 using FeatureUnion. Word features capture academic phrases; character features provide shorter patterns that can help with spelling and code-mixed forms. Three classifiers contribute probability estimates through weighted soft voting. [3, P3]

Estimator	Key setting	Vote weight
Calibrated LinearSVC	C = 1.5; calibration cv = 3	2
Logistic Regression	C = 5.0; balanced class weights	2
Complement Naive Bayes	alpha = 0.3; norm = True	1

Table 4.2: Intent ensemble configuration

For an intent c, the ensemble score is (2Psvc(c) + 2Plr(c) + Pnb(c)) / 5. The largest score gives the predicted class. The inference wrapper can return clarification when confidence falls below its configured threshold; this threshold is a routing choice, not an answer-accuracy guarantee.

4.4 Evidence Mapping and Worked Example

Field	Verified value
Question	What is the course code for Operating Systems?
Intent and entity	course_code; Operating Systems; CSE
Source	B.Tech. - Computer Science and Engineering.pdf, page 28
Source row	Operating Systems | 241CS013 | Semester IV
Supported answer	The course code for Operating Systems is 241CS013.

Table 4.3: Worked example checked against the original PDF

This example illustrates the mapping from question to source row. It is a manually checked evidence example, not a new measured execution of the chat service.

CHAPTER 5
ANSWER GENERATION VALIDATION AND EXECUTION

5.1 Retrieval and Reading-Comprehension Approach

CampusAI uses structured retrieval and evidence-based generation rather than a separately fine-tuned span-extraction reader. Exact academic intents can query SQL records. Descriptive requests use BM25 lexical search and dense semantic similarity over curriculum passages. The retriever combines candidate rankings through reciprocal rank fusion.

The fusion score for a passage d is the sum of w/(60 + rank(d)) across ranking lists in which it appears. Rank fusion combines positions rather than treating raw BM25 and vector scores as directly comparable. The configured dense model is all-MiniLM-L6-v2; the project also contains an optional cross-encoder reranker. [2, 4, P5]

5.2 Answer Generation

Pathway	Input evidence	Output
Structured facts	Course or semester records	Direct academic facts with provenance
Hybrid retrieval	Ranked curriculum passages	Explanation or summary based on evidence
Clarification	Ambiguous or uncertain query	A request to identify the intended entity
Abstention	Unsupported live schedule request	Referral to the official examination source
Social response	Greeting or thanks intent	Short conversational response

Table 5.1: Answer pathways and outputs

The answer generator constructs a prompt containing the query, selected passages, citations and optional conversation history. A configurable adapter generates a response. A mock adapter supports local demonstrations, while remote or local model providers require separate setup. The original RAG literature motivates retrieval-conditioned generation; CampusAI applies this pattern without claiming to train the original end-to-end RAG architecture. [1]

5.3 Validation and AI Response Handling

FastAPI and Pydantic validate query types and length limits. The versioned chat service returns answer text, intent, domain, language, extracted entities, citations, source type and a grounding flag. Source metadata allows the interface to open the original document and page.

Citation and attribution checks use identifiers, source text and heuristic scores. Such checks can show that a response refers to retrieved material, but they do not prove that every statement is entailed by that material. The grounding flag must therefore be interpreted as an internal system indicator.

5.4 Request Execution and Bounded Failover

The generation layer is configurable and the retrieval components can operate with saved local indices. Exact behaviour depends on the chosen route, available database, loaded models and provider configuration. Provider failures and absent indices need explicit test coverage before deployment; the existence of adapter code does not establish universal recovery behaviour.

5.5 Handling of Different Outcomes

Condition	Expected response behaviour
Course and source match	Return the relevant fact and its source page.
Ambiguous abbreviation	Offer alternatives and request clarification.
No suitable curriculum evidence	Explain that verified information was not located.
Tomorrow's examination schedule	Refer to the examination cell or official student portal.
Conflicting numerical extraction	Check the original table and flag the extracted value.
Citation present but answer off-topic	Treat as a retrieval or generation error, not a correct answer.

Table 5.2: Response outcomes and interpretation

Evidence blocks are treated as retrieved data rather than higher-level instructions. This is a useful prompt design measure, but a broader adversarial test suite is needed to substantiate resistance to prompt injection.

CHAPTER 6
IMPLEMENTATION

6.1 User Question Input

The React application maintains views for home, chat, course exploration, semesters, comparison and administration. Its API client sends the student question, optional session identifier, department preference and language preference to the versioned chat route. The backend creates or retrieves a chat session and persists messages and response citations.

Method and route	Purpose
POST /api/v1/chat	Conversational question answering and session persistence
GET /api/v1/departments	List academic departments
GET /api/v1/courses	Filter and search course records
GET /api/v1/courses/{course_id}	Retrieve detailed course information
GET /api/v1/semesters	Retrieve department and semester information
POST /api/chat	Additional core pipeline endpoint
POST /api/query	Compact output from the core pipeline
GET /api/health	Service status response

Table 6.1: Principal API routes inspected in the project

6.2 Preprocessing Implementation

backend/nlp/preprocessing.py provides the text cleaner, tokenizer and protected-term handling used by the training workflow. The classifier applies the same convenience preprocessor before inference. Related normalisation modules handle academic aliases and contextual query forms in the conversational route.

A consistent preprocessing contract is important: a model evaluated on one representation should not silently receive a different representation in deployment. Tests should include course codes, Roman numerals, Telugu-script queries and code-mixed questions.

6.3 Question Understanding Implementation

scripts/train_intent_model.py builds the dual-feature ensemble, performs five-fold stratified cross-validation and then evaluates a stratified 80:20 train-test split using random seed 42. The 1,201-row dataset yields 960 training queries and 241 test queries. It saves a unified pipeline, class labels, label encoder, metric summaries and a confusion matrix.

backend/nlp/intent_classifier.py loads the saved pipeline when present. Its own training method constructs a simpler TF-IDF and Logistic Regression baseline. Therefore, regenerating the model through that method is not equivalent to rerunning the ensemble training script. Model provenance and the entry point used for training must be retained.

6.4 AI Explanation Implementation

backend/rag/generator.py contains AnswerGenerator and ConversationalRAGEngine. The generator builds a prompt from selected evidence and calls an LLM adapter. The conversational engine adds language identification, ambiguity handling, context rewriting, multi-intent processing and citation checks. A separate StudentChatbotPipeline in backend/pipeline.py includes guardrail and attribution stages.

6.5 Important Code Paths

Path	Responsibility
scripts/train_intent_model.py	Ensemble training, validation and artefact export
backend/nlp/intent_classifier.py	Saved pipeline loading and intent prediction
backend/rag/retriever.py	SQL, sparse, dense and fused retrieval
backend/rag/generator.py	Conversation orchestration and answer generation
campusai/api/routes/chat.py	Versioned chat API and message persistence
frontend/src/api.ts	Browser API requests and response handling

Table 6.2: Implementation files and responsibilities

Selected source excerpts are included in the appendices. They document the implemented approach; they are not a substitute for the complete modules and their runtime dependencies.

Detailed integration: Appendix G (pages 30 onward) provides source listings, library imports, data contracts and explanations for NLP preprocessing, intent learning, semantic retrieval, reranking and language generation.

CHAPTER 7
RESULT VISUALIZATION AND DASHBOARD

7.1 Dashboard Design

The interface provides an assistant for natural-language questions and structured views for discovering courses, inspecting semesters and comparing departments. Course and source dialogs make detailed information available without requiring a student to leave the current task. Light and dark themes and responsive layouts are represented in the saved interface captures.

7.2 Dashboard Statistics

Corpus counts, result counts and response metadata serve different purposes. The catalogue result count reflects the active filter and available API data. Model accuracy comes from an offline evaluation file. A latency shown for one answer is a per-request measurement, not the system-wide mean.

7.3 Result Presentation Types

View	Information shown
Assistant	Question, answer, source citations and evidence status
Course catalogue	Course title, code, credits and filter controls
Course details	Syllabus, outcomes, prerequisites and reference material
Semester view	Department-specific course offerings and credit summary
Comparison view	Parallel department and semester information
Administration	Source documents, extraction issues and evaluation information

Table 7.1: Interface views and result presentation

The frontend stores the current tab and theme preferences and provides source-opening controls. Source inspection is especially valuable when the answer depends on an extracted numerical field. For example, a semester summary must distinguish total credits from total hours rather than presenting either number as interchangeable.

The screenshots on pages 19-20 are saved UI verification captures. Their fixture values illustrate layout and interaction states; they are not additional evidence of live academic-data accuracy.

7.4 Charts and Graphs



Figure 7.1: Saved classifier test metrics on 241 queries

The saved run dated 22 October 2026 reports 96.27% top-1 accuracy and 99.17% top-3 accuracy. Top-3 accuracy measures whether the correct label occurs among the three highest-scoring predictions; it is not the accuracy of the single label returned to the user. Five-fold cross-validation accuracy is 96.00%, with a standard deviation of 1.17 percentage points. [P3]

7.5 Sample Questions and Expected Outcomes

Question	Intended outcome
What is the course code for Operating Systems?	Identify 241CS013 and cite the CSE source row.
List CSE third-semester subjects.	Retrieve the department-specific semester offerings.
Explain Unit 1 of Data Structures.	Retrieve the correct course and unit passage.
Tell me about DS.	Clarify Data Structures versus Data Science.
What is tomorrow's exam schedule?	Abstain from inventing a live timetable.
Compare CSE and IT Semester 3.	Retrieve both semester structures before comparison.

Table 7.2: Demonstration questions and intended outcomes

These are demonstration expectations. They should be evaluated against independently checked answers and relevant source pages, rather than accepted solely because the predicted intent is correct.

Classifier Error Analysis

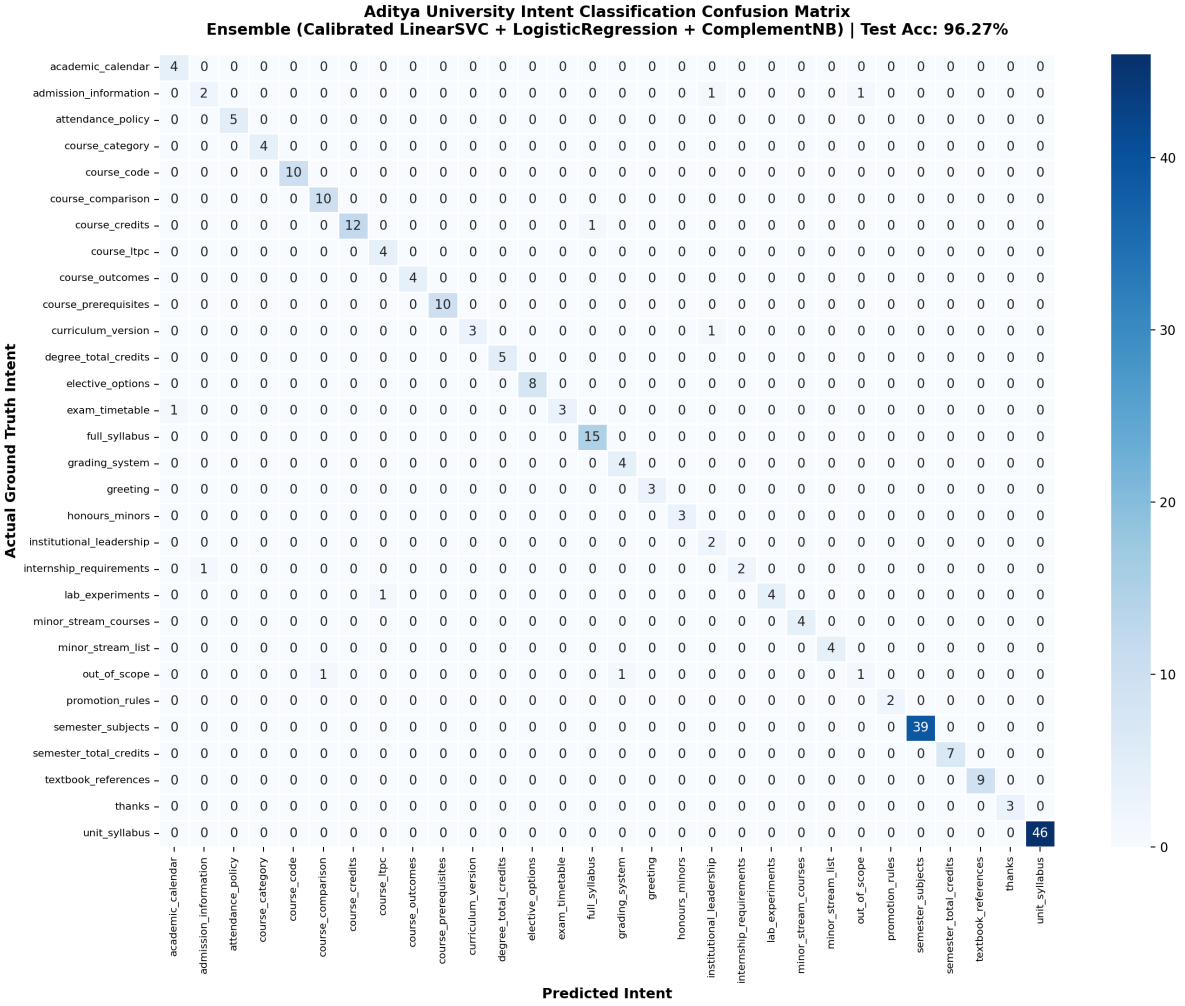


Figure 7.2: Saved confusion matrix for the 30-intent classifier

The classification report totals 241 test queries. The recorded accuracy corresponds to 232 correct predictions and 9 errors. Strong results for frequently represented syllabus and semester intents coexist with weaker results for some small classes. [P3]

Intent	Test support	F1 score
out_of_scope	3	0.4000
admission_information	4	0.5714
institutional_leadership	2	0.6667
internship_requirements	3	0.8000

Table 7.3: Selected lower-performing intent classes

Small test supports make per-class estimates unstable. Additional diverse examples and template-grouped evaluation are needed before these scores can be interpreted as general performance on new student questions.

7.6 Demonstration and Interface States

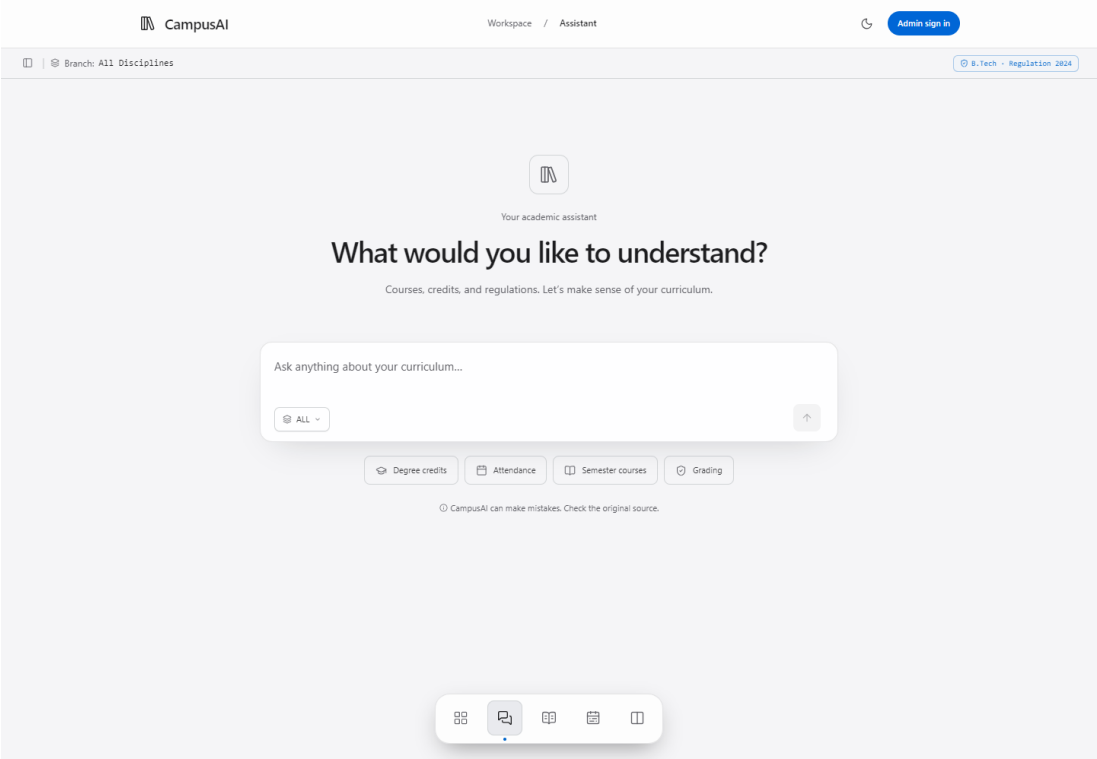


Figure 7.3: Assistant interface in a saved UI verification capture

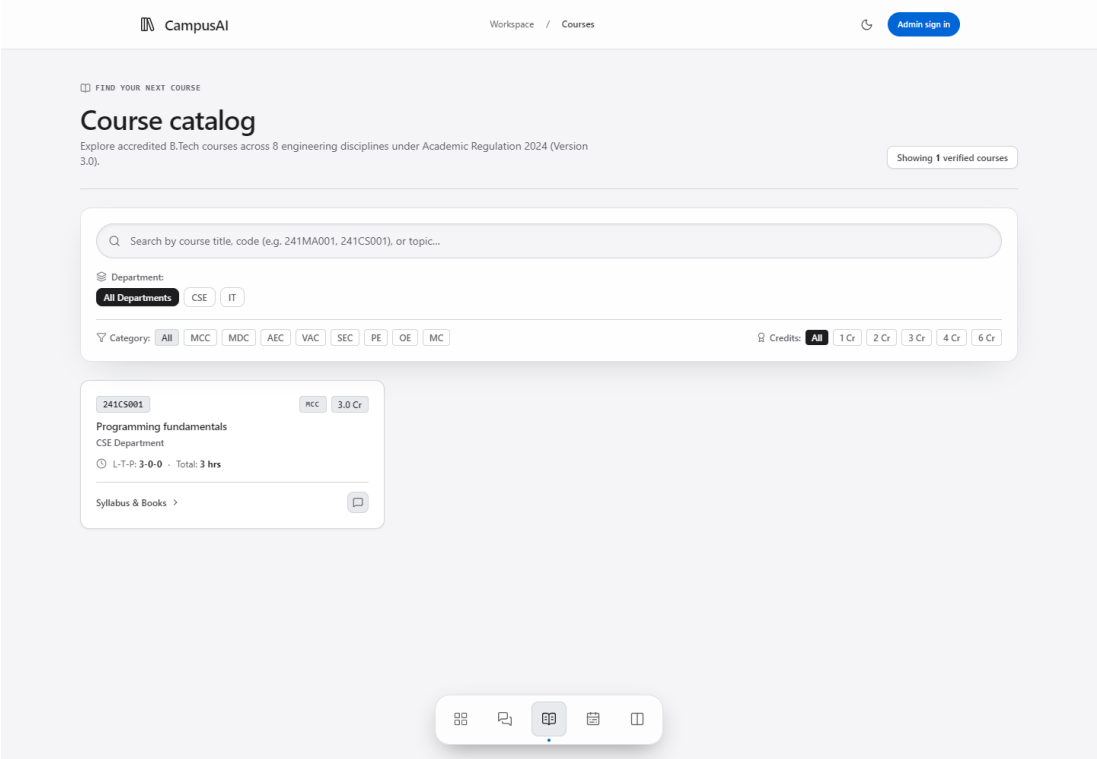


Figure 7.4: Course catalogue in a saved UI verification capture

These captures illustrate the question composer, navigation, filters and course cards. The saved verification uses controlled data; displayed record counts are not the size of the full corpus.

7.6 Demonstration and Interface States continued

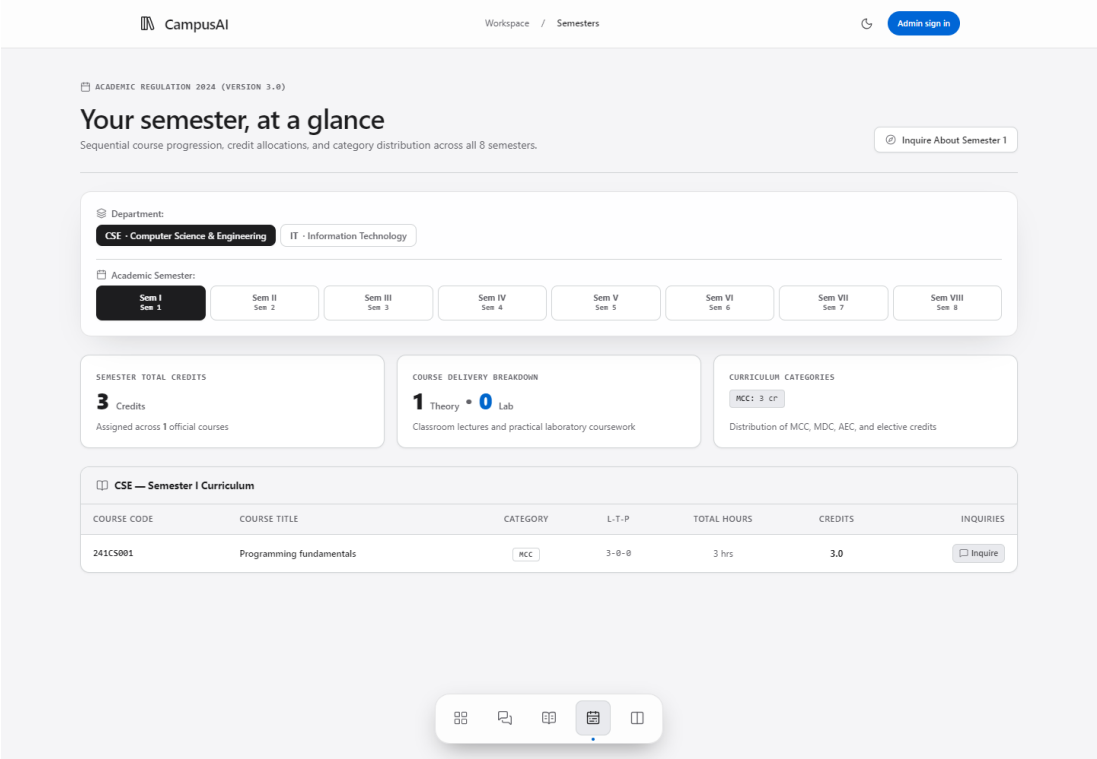


Figure 7.5: Semester view in a saved UI verification capture

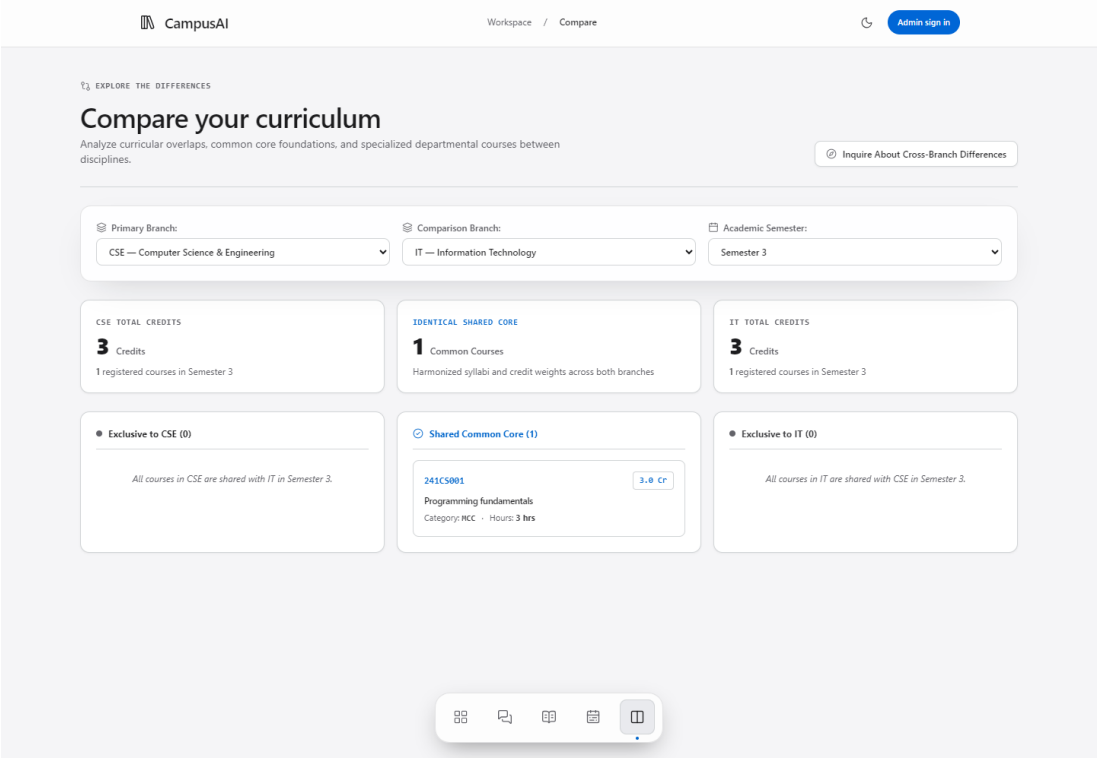


Figure 7.6: Comparison view in a saved UI verification capture

The semester and comparison screens support structured browsing alongside chat. The original curriculum should be opened when checking credits, contact hours or conflicting extracted fields.

7.7 Results and Discussion

Evaluation	Recorded result	Interpretation
Intent classifier	96.27% top-1; macro F1 0.9196	241 test queries; separate from answer correctness.
System intent match	16/16 = 100%	Small, curated benchmark dated 21 October 2026.
Academic fact checks	8/8 = 100%	Substring checks for expected values, not full semantic verification.
Grounding flags	15/16 = 93.75%	Internal flags; safe abstention is marked false in this run.
Latency	Mean 641.48 ms; p95 3,848.14 ms	Historical benchmark conditions; not a new timing run.
Retrieval quality scores	Recall@5 0.964; MRR 0.923	Hard-coded in the runner; not measured retrieval results.

Table 7.4: Recorded evaluations and their limits

The 16-query run predates the 22 October classifier report. Its figures must not be combined into a claim that the latest model achieves 100% end-to-end accuracy. The benchmark checks expected values by searching for strings in answers; a matching digit alone can pass an inadequate answer. [P4]

Some saved answer previews illustrate this limitation. The weather query retrieves hydrology-related curriculum text, and the academic-calendar query begins with a software-architecture passage, despite positive internal grounding flags. These examples show that citation presence and intent matching are insufficient to establish relevance.

Data integrity is another constraint: the CSE Semester IV summary confuses 32 hours with 20 credits. Correct retrieval of an erroneous passage can still produce an incorrect answer. Evaluation must test extraction, retrieval relevance, answer completeness and source faithfulness separately.

Saved frontend reports record 37/37 checks in the earlier verification file and 56/56 checks in the redesign report. These are historical UI checks with controlled test states; they are not newly executed backend regression results. [P6]

CHAPTER 8
CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

CampusAI integrates a curriculum evidence corpus, academic entity handling, a trained intent ensemble and multiple retrieval paths into a student-facing application. The verified local artefacts contain eight curriculum PDFs, 1,039 passages and 1,201 labelled questions. The saved classifier achieves 96.27% top-1 accuracy on 241 test queries, providing a useful routing component for the helpdesk.

The strongest contribution is the connection between natural-language interaction and inspectable academic sources. The evaluation also demonstrates why classifier scores, source references and answer correctness must remain separate: an accurate intent label does not repair an incorrectly extracted table or an irrelevant retrieved passage.

8.2 Limitations

The query dataset is imbalanced and project-curated. Similar question templates can occur across random train-test partitions, even when exact duplicate strings are absent. Small classes have limited test support. The saved system benchmark is small, uses substring fact checks and includes retrieval constants rather than computed relevance metrics.

The current evidence also contains extraction errors and uneven passage coverage across departments. Multilingual and conversational functionality require dedicated evaluation. Historical interface tests and screenshots do not establish a verified production deployment or comprehensive academic correctness.

8.3 Future Enhancements

Priority improvements are to validate credits against original table columns, add human-reviewed relevance judgements, compute retrieval metrics from ranked results, and evaluate complete answers independently. A template-grouped split and new questions from students would better assess generalisation.

Further work should expand low-support intents, test Telugu and code-mixed queries separately, align the two orchestration routes, version datasets and model artefacts together, strengthen session access controls, and verify the freshness of official notices before using them in answers.

REFERENCES

[1] Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* arXiv:2005.11401. https://arxiv.org/abs/2005.11401

[2] Reimers, N. and Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* arXiv:1908.10084. https://arxiv.org/abs/1908.10084

[3] scikit-learn documentation. *Voting Classifier.* Probability-based soft voting and weighted ensembles. Accessed 25 October 2026. scikit-learn.org/stable/modules/ensemble.html

[4] Sentence Transformers. *all-MiniLM-L6-v2 model card.* Model description and embedding dimensions. Accessed 25 October 2026. huggingface.co/sentence-transformers/all-MiniLM-L6-v2

Project Evidence Sources

[P1] Local corpus and retrieval artefacts: data/institutional/curriculum/ (eight PDFs); data/manifests/source_quality_report.json; data/indices/chunks_metadata.json; data/indices/dense_embeddings.npy. PDF pages and index dimensions inspected on 25 October 2026.

[P2] Query data: data/aditya_university_queries.csv; data/evaluations/intent_dataset.json; data/structured/intents.json; scripts/build_curriculum_dataset.py. Row counts, languages and exact duplicates inspected on 25 October 2026.

[P3] Classifier evaluation: reports/model_metrics.json; reports/classification_report.txt; reports/confusion_matrix.png; scripts/train_intent_model.py. Saved model report dated 22 October 2026.

[P4] System benchmark: data/evaluations/benchmark_results.json, dated 21 October 2026; backend/evaluation/benchmark_runner.py. Read alongside per-query answer previews and metric implementation.

[P5] Implementation: backend/nlp/, backend/rag/, backend/pipeline.py, backend/main.py, backend/database/models.py, campusai/api/routes/chat.py, frontend/src/api.ts and frontend/src/App.tsx.

[P6] Interface verification: frontend/qa/verification.json; frontend/qa/redesign/report.json; saved screenshots in frontend/qa/redesign/. These are existing project artefacts, not a fresh execution.

[P7] Worked academic example: B.Tech. - Computer Science and Engineering.pdf, printed page 28; source row for 241CS013 and Semester IV total fields.

APPENDIX A
SOURCE CODE FOR INTENT FEATURES

Selected complete feature-extractor and ensemble-builder functions from scripts/train_intent_model.py. These functions implement the architecture associated with the saved 30-intent evaluation. Long lines may wrap for print.

```
def build_feature_extractor() -> FeatureUnion:
    """Build dual-channel feature extractor fusing word n-grams and character subwords."""
    return FeatureUnion([
        ("word_tfidf", TfidfVectorizer(
            ngram_range=(1, 3),
            min_df=1,
            max_df=0.90,
            sublinear_tf=True,
            token_pattern=r"(?u)\b\w+\b"
        )),
        ("char_tfidf", TfidfVectorizer(
            ngram_range=(2, 5),
            min_df=2,
            max_df=0.90,
            sublinear_tf=True,
            analyzer="char_wb"
        ))
    ])

def build_ensemble_classifier(random_state: int = 42) -> VotingClassifier:
    """Construct calibrated multi-algorithm soft-voting ensemble."""
    base_svc = LinearSVC(C=1.5, loss="squared_hinge", penalty="l2", random_state=random_state)
    cal_svc = CalibratedClassifierCV(base_svc, cv=3)
    lr = LogisticRegression(
        C=5.0,
        max_iter=1000,
        solver="lbfgs",
        class_weight="balanced",
        random_state=random_state
    )
    cnb = ComplementNB(alpha=0.3, norm=True)

    return VotingClassifier(
        estimators=[
            ("cal_svc", cal_svc),
            ("logistic_reg", lr),
            ("complement_nb", cnb)
        ],
        voting="soft",
        weights=[2.0, 2.0, 1.0]
    )
```

APPENDIX B
SOURCE CODE FOR PREPROCESSING

Selected methods and the convenience entry point from backend/nlp/preprocessing.py. Protected terms and the default preprocessor object are defined in the same module. This excerpt preserves course-code handling and the lemmatisation fallback.

```
def clean_text(self, text: str) -> str:
    """Lowercase and normalize quotation marks and dashes."""
    if not text:
        return ""
    text = text.lower()
    text = re.sub(r"[\u2018\u2019]", "'", text)
    text = re.sub(r"[\u201c\u201d]", '"', text)
    text = re.sub(r"[\u2013\u2014]", "-", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

def tokenize(self, text: str) -> List[str]:
    """Tokenize preserving alphanumeric course codes, unit specifications, acronyms, and
        Telugu script."""
    cleaned = self.clean_text(text)
    tokens = re.findall(r"\b[a-z0-9\u0C00-\u0C7F]+(?:-[a-z0-9\u0C00-\u0C7F]+)*\b", cleaned)
    return tokens

def preprocess(self, text: str) -> str:
    """Full pipeline: clean -> tokenize -> domain filter -> lemmatize -> join."""
    tokens = self.tokenize(text)
    processed_tokens: List[str] = []

    for token in tokens:
        if token in PROTECTED_ENTITIES or re.match(r"^241[a-z]{2}[0-9]{3}$", token):
            processed_tokens.append(token)
            continue

        if self.remove_stopwords and token in STOP_WORDS:
            continue

        if self.lemmatize and lemmatizer is not None:
            try:
                lemma = lemmatizer.lemmatize(token)
            except Exception:
                lemma = token
            processed_tokens.append(lemma)
        else:
            processed_tokens.append(token)

    return " ".join(processed_tokens)

def preprocess_query(query: str) -> str:
    """Convenience helper for query preprocessing."""
    return preprocessor.preprocess(query)
```

APPENDIX C
SOURCE CODE FOR QUERY HANDLING

NLP query handling: selected course-code and semester extraction logic from AcademicEntityExtractor.extract_entities in backend/nlp/entity_extractor.py. This is rule-based entity recognition, not a trained named-entity recognition model.

Imports: re; normalize_query; ROMAN_NUMERALS; WORD_NUMBERS

```
145         code_match = re.search(r"\b(241[A-Z]{2}[0-9]{3})\b", normalized)
146         if code_match:
147             entities["course_code"] = code_match.group(1).upper()
148             entities["course"] = code_match.group(1).upper()
149         else:
150             short_code_match = re.search(r"\b([A-Za-z]{2,3})\s*[-]?\s*(\d{3})\b", text)
151             if short_code_match:
152                 raw_code = f"{short_code_match.group(1).upper()}{short_code_match.group(2)}"
153                 if raw_code in self.code_to_course:
154                     entities["course_code"] = raw_code
155                     entities["course"] = raw_code
156 
157         # 2. Semester Number
158         sem_match = re.search(r"\b(?:semester|sem)\s*[-]?\s*([1-8]|i{1,3}|iv|v|vi{1,3}|viii)\b",
  |  lower)
159         if sem_match:
160             val = sem_match.group(1)
161             entities["semester"] = int(val) if val.isdigit() else ROMAN_NUMERALS.get(val)
162         else:
163             ord_match = re.search(r"\b(1st|2nd|3rd|4th|5th|6th|7th|8th|first|second|third|fourth|fif
  |  th|sixth|seventh|eighth)[\s\-]+(?:semester|sem)\b", lower)
164             if ord_match:
165                 v = ord_match.group(1)
166                 entities["semester"] = int(v[0]) if v[0].isdigit() else WORD_NUMBERS.get(v)
167 
168         # 3. Unit Number
169         unit_match = re.search(r"\b(?:unit|module)\s*[-]?\s*([1-5]|i{1,3}|iv|v)\b", lower)
```

Integration with NLP

normalize_query supplies a consistent text form before regular-expression matching. Course-code patterns detect academic identifiers; code_to_course validates shorter identifiers against the loaded vocabulary. Semester expressions map digits, Roman numerals and ordinal words into an integer slot.

The surrounding method also extracts units, departments and course names. These entity slots constrain retrieval after intent classification. The dictionaries and method context are defined in the complete source module.

APPENDIX D
INTENT DATASET DISTRIBUTION

Counts below are calculated directly from the saved 1,201-row query CSV. Each row has one exported primary intent.

Intent	Count	Intent	Count
academic_calendar	18	grading_system	22
admission_information	20	greeting	15
attendance_policy	24	honours_minors	15
course_category	20	institutional_leadership	12
course_code	49	internship_requirements	17
course_comparison	48	lab_experiments	27
course_credits	65	minor_stream_courses	20
course_ltpc	20	minor_stream_list	18
course_outcomes	19	out_of_scope	15
course_prerequisites	48	promotion_rules	13
curriculum_version	18	semester_subjects	197
degree_total_credits	26	semester_total_credits	35
elective_options	38	textbook_references	46
exam_timetable	18	thanks	15
full_syllabus	74	unit_syllabus	229

Language totals: English 850 (70.77%), Telugu 48 (4.00%), and Telugu-English 303 (25.23%). These are dataset labels, not independently audited language-identification accuracy.

The script performs stratified random splitting, which preserves approximate class proportions. It does not group paraphrases by template or source entity. A stronger evaluation would hold out question patterns and evaluate new student questions separately.

No exact query duplicates were found. This does not rule out near-duplicate phrasing or semantically equivalent examples in different partitions.

APPENDIX E
REPRODUCIBILITY AND RESULT PROVENANCE

E.1 Keep Evaluation Artefacts Separate

reports/model_metrics.json and reports/classification_report.txt describe the 30-intent ensemble. reports/evaluation_results.json describes an older 25-class run with 757 samples and 92.11% accuracy. These runs differ in dataset size and label space, so their scores do not constitute a controlled model comparison.

data/evaluations/benchmark_results.json describes the separate 16-query conversational benchmark. Its saved timestamp precedes the ensemble report. A new system-level claim requires rerunning the relevant route with the intended model, data and provider configuration.

E.2 Required Validation Before Academic Use

Check	Evidence to retain
Dataset identity	CSV checksum, class counts, source documents and version
Split integrity	Random seed, split indices and template-grouping policy
Model identity	Training script, package versions and saved pipeline checksum
Extraction quality	Original PDF rows compared with credits and hours fields
Retrieval quality	Human relevance labels and ranked passage identifiers
Answer quality	Independent expected answers and citation-entailment review
Runtime conditions	API route, database state, model provider and timing environment

Table E.1: Reproducibility and validation checks

E.3 Report Verification

The corpus page counts, query distribution, index dimensions and worked PDF example were inspected for this report. The classifier and system metrics are taken from saved project evaluations; no new model training or full benchmark execution is claimed. The report separates intended behaviour, source-code evidence, archived UI states and recorded numerical results.

APPENDIX F
REQUIREMENTS AND RUN INSTRUCTIONS

F.1 NLP and ML Dependencies

The project requirements include pandas, scikit-learn, NumPy, Sentence Transformers, rank-bm25 and PyMuPDF. Training and preprocessing additionally use NLTK, Matplotlib, Seaborn and joblib. Appendix G explains each integration. The commands below operate on the existing project environment.

```
python -m pip install -r requirements.txt
python -m pip install nltk matplotlib seaborn joblib
```

F.2 Preserve Training Artefacts

Keep the labelled question CSV, preprocessing source, feature builders, random seed and class labels together. Use separate output directories to retain the current trained model and saved evaluation.

```
python scripts/train_intent_model.py --models-dir tmp/nlp_models
    --reports-dir tmp/nlp_reports
```

Run the command as one line in PowerShell; the displayed continuation separates its arguments for print. This command trains the intent classifier and writes new artefacts. It was not executed to prepare this report.

F.3 Verify Model Compatibility

Use the same text preprocessing at inference. Check the saved label ordering before interpreting predict_proba output. Keep sentence-embedding matrices aligned with their passage metadata, and regenerate the index when the embedding model changes.

F.4 Interpret the Results

Compare new model metrics using the same dataset and split policy. Use independently labelled passages to evaluate retrieval, and inspect answers against the original curriculum. Saved classifier metrics do not establish complete answer correctness.

APPENDIX G
DETAILED NLP AND ML CODE

This appendix extends Chapter 6 and Appendices A-B with the NLP and machine-learning implementation used in CampusAI. It follows curriculum text and student questions through preprocessing, intent learning, embeddings, hybrid retrieval, reranking and evidence-based generation.

The listings are source excerpts from the current project. Printed line numbers identify their location in the named file. A vertical bar marks a wrapped continuation of a long source line. Excerpts may depend on module imports, settings, model classes or surrounding function variables and should be read with the complete source file.

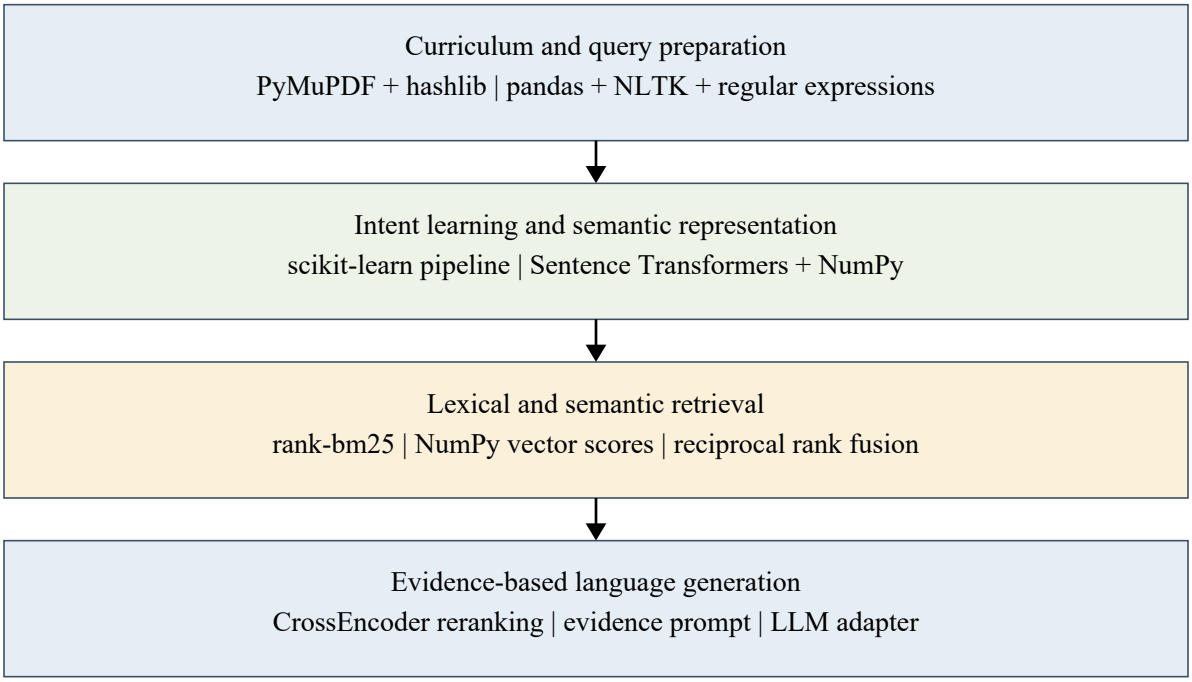


Figure G.1: NLP and ML library integration pipeline

Appendix navigation

The following pages explain NLP dependencies, text extraction, model preparation, fitting, model loading, retrieval, reranking and prompt construction. Appendices A and B contain the detailed TF-IDF, ensemble and preprocessing functions; Appendix C covers rule-based academic entity extraction. Only NLP and ML source code is reproduced.

APPENDIX G CONTENTS

G.1 Python library integration 32

G.2 NLP supporting libraries 33

G.3 PDF Text Extraction and Source Inventory 34

G.4 Training Data and Feature Pipeline 35

G.5 Model Fitting and Test Metrics 36

G.6 Model Persistence and Inference 38

G.7 Lexical Retrieval with BM25 40

G.8 Sentence Embeddings with NumPy 41

G.9 Dense Similarity Search 42

G.10 Combining Lexical and Semantic Rankings 43

G.11 Retrieval Orchestration and Output Mapping 44

G.12 Optional Cross Encoder Reranking 45

G.13 Evidence Prompt Construction 46

G.14 Generation Adapter Selection 47

G.15 NLP and ML data flow 48

G.16 Documentation and code provenance 49

G.1 Python Library Integration

The versions below reproduce declarations in requirements.txt. They are minimum constraints, not a verified list of installed versions. Imports and usage were checked against the project source.

Library and declaration	Import or API	Integration role
pymupdf >= 1.25.0	import fitz	Open curriculum PDFs and extract page text.
pandas >= 2.2.0	pd.read_csv	Load labelled questions into a DataFrame.
scikit-learn >= 1.5.0	Pipeline; FeatureUnion	Fit TF-IDF features and the ensemble together.
numpy >= 1.26.0	np.array; np.dot	Store vectors, compute similarity and sort probabilities.
sentence-transformers >= 3.3.0	SentenceTransformer; CrossEncoder	Encode text; optionally rerank query-passage pairs.
rank-bm25 >= 0.2.2	BM25Okapi	Score tokenised passages against query terms.

Table G.1: Python libraries and their integration roles

NLTK, Matplotlib and Seaborn are imported by preprocessing or training code but are not explicitly listed in requirements.txt. joblib is used for model artefacts and is commonly installed through scikit-learn, but an explicit direct dependency is preferable for a reproducible project environment.

G.2 NLP Supporting Libraries

Dependency	Import	NLP or ML purpose
NLTK	WordNetLemmatizer	Lemmatise unprotected tokens; fall back to the token when unavailable.
joblib and pickle	joblib; pickle	Persist and load trusted fitted estimators and metadata.
Matplotlib and Seaborn	pyplot; seaborn	Plot model evaluation and the confusion matrix.
Regular expressions	re (standard library)	Normalise text, preserve course codes and identify academic entities.
Provider SDKs	google.generativeai; openai	Optional language generation through the configured adapter.

Table G.2: Supporting NLP and ML dependencies

Preprocessing Integration

Appendix B reproduces clean_text, tokenize, preprocess and preprocess_query from backend/nlp/preprocessing.py. Python regular expressions retain alphanumeric course codes and Telugu characters. The domain vocabulary protects academic abbreviations from lemmatisation. Stop-word removal is disabled in the default preprocessor.

NLTK WordNetLemmatizer is optional in the current implementation. If the import fails or its data resource is unavailable, the original token is retained. Training and prediction must use the same preprocessing settings to keep their feature representation consistent.

Model Integration

Appendix A contains the FeatureUnion and soft-voting ensemble builders. Word and character TF-IDF representations are combined before the classifier. The classifier combines calibrated LinearSVC, Logistic Regression and Complement Naive Bayes. The following listings show how those builders connect to data loading, validation, model fitting and prediction.

The generation SDKs are imported within the selected adapter. The factory supports configured Gemini or OpenAI generation, with deterministic offline generation as the fallback. These are generation dependencies; they do not train the intent classifier.

G.3 PDF Text Extraction and Source Inventory

The inventory loop connects file metadata, a checksum and PyMuPDF text extraction. It samples pages to determine whether a PDF has extractable text before creating the manifest record.

Source: backend\data_ingestion\inventory.py
Library imports: import fitz; import hashlib; from pathlib import Path

```
180
181         for idx, pdf_path in enumerate(pdf_files, 1):
182             file_size = pdf_path.stat().st_size
183             sha256 = compute_sha256(pdf_path)
184
185             doc = fitz.open(pdf_path)
186             page_count = len(doc)
187
188             pages_with_text = 0
189             sample_size = min(20, page_count)
190             for pno in range(sample_size):
191                 if len(doc[pno].get_text().strip()) > 30:
192                     pages_with_text += 1
193             text_ratio = round(pages_with_text / max(1, sample_size), 3)
194             doc.close()
195
196             doc_reopened = fitz.open(pdf_path)
197             meta = extract_frontmatter_metadata(doc_reopened, pdf_path.name)
198             doc_reopened.close()
199
200             doc_id = f"DOC-{meta['department_code']}-{idx:02d}"
```

How the libraries connect

fitz.open returns a document; len(doc) supplies its page count and get_text returns page text. The checksum helper reads file bytes and hashes them with SHA-256. Metadata extraction then reads the front matter and identifies department and curriculum information. The text ratio is based on at most 20 sampled pages, so it is not a complete extraction-quality score. [L5]

Input and output: PDF path -> file size, checksum, page count, sampled text ratio and front-matter metadata. These fields feed DocumentMetadata and the JSON/CSV manifests.

G.4 Training Data and Feature Pipeline

This section connects pandas with the domain preprocessor, LabelEncoder and a scikit-learn Pipeline. The feature and classifier builder functions are printed in Appendix A.

Source: scripts\train_intent_model.py
Library imports: import pandas as pd; LabelEncoder; Pipeline; StratifiedKFold; cross_val_score

```
124      # 1. Load and Clean Dataset
125      df = pd.read_csv(data_path)
126      logger.info(f"Loaded {len(df):,} queries across {df['intent'].nunique()} intents.")
127
128      # 2. Domain-Aware Preprocessing
129      logger.info("Applying domain-aware query normalization...")
130      X_raw = df["query"].fillna("").astype(str).tolist()
131      y_raw = df["intent"].astype(str).tolist()
132
133      X_preprocessed = [preprocess_query(q) for q in X_raw]
134
135      # Label encoding
136      label_encoder = LabelEncoder()
137      y_encoded = label_encoder.fit_transform(y_raw)
138      classes = list(label_encoder.classes_)
139      num_classes = len(classes)
140
141      # 3. Stratified K-Fold Cross-Validation
142      logger.info(f"Running {cv_folds}-Fold Stratified Cross-Validation on Full Dataset...")
143      full_pipeline = Pipeline([
144          ("features", build_feature_extractor()),
145          ("ensemble", build_ensemble_classifier(random_state=seed))
146      ])
147
148      skf = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state=seed)
149      cv_scores = cross_val_score(full_pipeline, X_preprocessed, y_encoded, cv=skf,
  |  scoring="accuracy")
```

How the libraries connect

pandas supplies the query and intent columns. preprocess_query normalises each string before feature extraction. LabelEncoder maps class names to integer labels. FeatureUnion concatenates word and character features, while Pipeline keeps feature fitting and classifier fitting inside each validation fold. [L1]

Input and output: CSV rows -> cleaned query list and encoded labels -> five validation scores from cloned fitted pipelines.

G.5 Model Fitting and Test Metrics

The evaluation path fits on the training partition and computes both predicted labels and class probabilities for held-out questions.

Source: scripts\train_intent_model.py
Library imports: train_test_split; accuracy_score; top_k_accuracy_score; f1_score; classification_report

```
153     logger.info(f"Splitting dataset: {100 * (1 - test_size):.0f}% Train / {100 * test_size:.0f}%
  |  Test...")
154     X_train, X_test, y_train, y_test = train_test_split(
155         X_preprocessed,
156         y_encoded,
157         test_size=test_size,
158         random_state=seed,
159         stratify=y_encoded
160     )
161     logger.info(f"Training split: {len(X_train):,} samples | Test split: {len(X_test):,} samples")
162
163     # 5. Model Fitting
164     logger.info("Fitting Dual Feature Union and Soft-Voting Ensemble on training split...")
165     fit_t0 = time.time()
166     full_pipeline.fit(X_train, y_train)
167     fit_duration = time.time() - fit_t0
168     logger.info(f"Model fitting completed in {fit_duration:.2f} seconds.")
169
170     # 6. Evaluation on Held-Out Test Split
171     logger.info("Evaluating ensemble model on held-out test set...")
172     y_pred = full_pipeline.predict(X_test)
173     y_probs = full_pipeline.predict_proba(X_test)
174
175     acc = float(accuracy_score(y_test, y_pred))
176     top3_acc = float(top_k_accuracy_score(y_test, y_probs, k=3, labels=list(range(num_classes))))
177     macro_f1 = float(f1_score(y_test, y_pred, average="macro", zero_division=0))
```

How the libraries connect

The split uses stratify=y_encoded to preserve approximate class proportions. fit trains the feature extractor and ensemble. predict provides top-1 labels; predict_proba provides the probability columns used by top-3 accuracy. Macro F1 gives equal weight to classes, while weighted F1 reflects test support. A random stratified split does not prevent similar templates crossing partitions.

Input and output: Cleaned texts and encoded labels -> trained pipeline -> predictions, probability matrix and test metric values.

G.5 Model Fitting and Test Metrics continued

Continuation of the same source excerpt. Cleaned texts and encoded labels -> trained pipeline -> predictions, probability matrix and test metric values.

Source: scripts\train_intent_model.py
Library imports: train_test_split; accuracy_score; top_k_accuracy_score; f1_score; classification_report

```
178      weighted_f1 = float(f1_score(y_test, y_pred, average="weighted", zero_division=0))
179      precision_w = float(precision_score(y_test, y_pred, average="weighted", zero_division=0))
180      recall_w = float(recall_score(y_test, y_pred, average="weighted", zero_division=0))
181
182      report_text = classification_report(
183          y_test,
184          y_pred,
185          labels=list(range(num_classes)),
186          target_names=classes,
187          digits=4,
188          zero_division=0
189      )
190      report_dict = classification_report(
191          y_test,
192          y_pred,
193          labels=list(range(num_classes)),
194          target_names=classes,
195          output_dict=True,
196          zero_division=0
197      )
198
199      logger.info("\n" + "=" * 70)
200      logger.info("HELD-OUT TEST SET EVALUATION REPORT")
201      logger.info("=" * 70)
202      for line in report_text.splitlines():
203          logger.info(line)
```

How the libraries connect

The split uses stratify=y_encoded to preserve approximate class proportions. fit trains the feature extractor and ensemble. predict provides top-1 labels; predict_proba provides the probability columns used by top-3 accuracy. Macro F1 gives equal weight to classes, while weighted F1 reflects test support. A random stratified split does not prevent similar templates crossing partitions.

Input and output: Cleaned texts and encoded labels -> trained pipeline -> predictions, probability matrix and test metric values.

G.6 Model Persistence and Inference

The inference wrapper loads a complete saved pipeline and uses its class list to interpret probability columns. The ensemble training script saves this pipeline together with label and metric metadata.

Source: backend\nlp\intent_classifier.py
Library imports: import pickle; import numpy as np; from backend.nlp.preprocessing import preprocess_query

```
152      def load_model(self) -> bool:
153          """Load trained model from disk."""
154          if not self.model_path.exists():
155              return False
156          with open(self.model_path, "rb") as f:
157              data = pickle.load(f)
158              self.pipeline = data["pipeline"]
159              self.classes_ = data["classes"]
160              self.metrics_ = data.get("metrics", {})
161              return True

163      def predict(self, text: str) -> IntentClassificationResult:
164          """Classify input query into an intent with confidence score."""
165          if self.pipeline is None:
166              if not self.load_model():
167                  self.train_and_evaluate()
168
169          from backend.nlp.preprocessing import preprocess_query
170          clean_text = preprocess_query(text)
171          probs = self.pipeline.predict_proba([clean_text if clean_text else text])[0]
172          top_indices = np.argsort(probs)[::-1]
173
```

How the libraries connect

pickle.load reconstructs the stored estimator graph. predict_proba runs the same feature transformations and classifier without fitting again. np.argsort orders the probabilities so the wrapper can select the highest-scoring class and alternatives. The threshold can change the returned intent to clarification. Load only trusted project model artefacts because pickle can execute code during loading.

Input and output: Question string -> IntentClassificationResult with predicted intent, confidence, domain, confidence flag and alternative intents.

G.6 Model Persistence and Inference continued

Continuation of the same source excerpt. Question string -> IntentClassificationResult with predicted intent, confidence, domain, confidence flag and alternative intents.

Source: backend\nlp\intent_classifier.py
Library imports: import pickle; import numpy as np; from backend.nlp.preprocessing import preprocess_query

```
174         best_idx = top_indices[0]
175         best_intent = self.classes_[best_idx]
176         confidence = float(probs[best_idx])
177 
178         alternatives = [
179             (self.classes_[i], float(probs[i]))
180             for i in top_indices[1:4]
181             if probs[i] > 0.05
182         ]
183 
184         is_confident = confidence >= self.confidence_threshold
185         final_intent = best_intent if is_confident else "clarification"
186 
187         domain = INTENT_TAXONOMY.get(final_intent, {}).get("domain", "curriculum")
188 
189         return IntentClassificationResult(
190             predicted_intent=final_intent,
191             confidence=round(confidence, 3),
192             domain=domain,
193             is_confident=is_confident,
194             alternative_intents=alternatives
195         )
```

How the libraries connect

pickle.load reconstructs the stored estimator graph. predict_proba runs the same feature transformations and classifier without fitting again. np.argsort orders the probabilities so the wrapper can select the highest-scoring class and alternatives. The threshold can change the returned intent to clarification. Load only trusted project model artefacts because pickle can execute code during loading.

Input and output: Question string -> IntentClassificationResult with predicted intent, confidence, domain, confidence flag and alternative intents.

G.7 Lexical Retrieval with BM25

BM25 uses academic tokens to score passages that contain the query terms. The full module contains academic_tokenizer, saved-index loading and JSON persistence.

Source: backend\rag\retriever.py
Library imports: from rank_bm25 import BM25Okapi; from dataclasses import asdict

```
44      def build_index(self, chunks: List[KnowledgeChunk]) -> None:
45          """Build BM25 index from knowledge chunks and save to disk."""
46          self.chunks = [asdict(c) for c in chunks]
47          tokenized_corpus = [academic_tokenizer(c["content"]) for c in self.chunks]
48          self.bm25 = BM25Okapi(tokenized_corpus, k1=settings.BM25_K1, b=settings.BM25_B)
49          self.save_index()

72      def search(
73          self, query: str, top_k: int = 10, dept_filter: Optional[str] = None
74      ) -> List[Dict[str, Any]]:
75          """Search BM25 index using tokenized query."""
76          if self.bm25 is None:
77              if not self.load_index():
78                  return []
79
80          tokens = academic_tokenizer(query)
81          if not tokens:
82              return []
83
84          scores = self.bm25.get_scores(tokens)
85          ranked = []
86          for idx, score in enumerate(scores):
87              chunk = self.chunks[idx]
88              if dept_filter and chunk.get("department_code") != dept_filter:
89                  continue
90              if score > 0.0:
91                  ranked.append({
92                      "chunk_id": chunk["chunk_id"],
93                      "score": float(score),
94                      "chunk": chunk
95                  })
96
97          ranked.sort(key=lambda x: x["score"], reverse=True)
98          return ranked[:top_k]
```

How the libraries connect

asdict converts knowledge chunks into serialisable dictionaries. The same academic tokenizer prepares indexed passages and the query. BM25Okapi builds corpus statistics; get_scores produces one score per passage. The code retains positive-scoring matches, applies the department filter and sorts candidates before selecting top_k. BM25 scores are relevance values, not probabilities.

Input and output: Question and optional department -> ranked dictionaries with chunk_id, score and the original chunk.

G.8 Sentence Embeddings with NumPy

The embedding service lazily creates its SentenceTransformer model and returns NumPy arrays. It is shared by the dense retriever.

Source: backend\rag\embeddings.py
Library imports: import numpy as np; from sentence_transformers import SentenceTransformer

```
11  class EmbeddingService:
12      """Manages transformer embedding generation with lazy model initialization."""
13
14      def __init__(self, model_name: Optional[str] = None):
15          self.model_name = model_name or settings.EMBEDDING_MODEL_NAME
16          self._model: Optional[SentenceTransformer] = None
17
18      @property
19      def model(self) -> SentenceTransformer:
20          if self._model is None:
21              self._model = SentenceTransformer(self.model_name, device=settings.DEFAULT_DEVICE)
22          return self._model
23
24      def encode(self, texts: List[str], batch_size: int = 64, normalize: bool = True) -> np.ndarray:
25          """Encode list of strings into numpy float32 matrix."""
26          if not texts:
27              return np.empty((0, 384), dtype=np.float32)
28          embeddings = self.model.encode(
29              texts,
30              batch_size=batch_size,
31              show_progress_bar=False,
32              normalize_embeddings=normalize
33          )
34          return np.array(embeddings, dtype=np.float32)
35
36      def encode_query(self, query: str) -> np.ndarray:
37          """Encode single query into 1D normalized vector."""
38          return self.encode([query], normalize=True)[0]
```

How the libraries connect

The model property delays weight loading until an embedding is requested. encode batches texts and requests normalised vectors, then converts the result to float32. encode_query extracts the first row for one question. The empty-array branch assumes 384 dimensions; changing the embedding model requires checking this assumption as well as rebuilding the saved index. [L2]

Input and output: List of texts -> float32 array of shape (number of texts, embedding dimensions); one query -> one normalised vector.

G.9 Dense Similarity Search

The dense retriever loads the vector matrix and passage metadata as a matched pair, then computes a score for every passage.

Source: backend\rag\retriever.py
Library imports: import numpy as np; import json; embedding_service.encode_query

```
114      def load_index(self) -> bool:
115          """Load precomputed embeddings and metadata."""
116          if not self.embeddings_file.exists() or not self.metadata_file.exists():
117              return False
118          try:
119              self.embeddings = np.load(self.embeddings_file)
120              with open(self.metadata_file, "r", encoding="utf-8") as f:
121                  data = json.load(f)
122                  self.chunks = data.get("chunks", [])
123              return True
124          except Exception:
125              return False

127      def search(
128          self, query: str, top_k: int = 10, dept_filter: Optional[str] = None
129      ) -> List[Dict[str, Any]]:
130          """Dense semantic search using cosine similarity."""
131          if self.embeddings is None:
132              if not self.load_index():
133                  return []
134
135          query_emb = embedding_service.encode_query(query)
136          sims = np.dot(self.embeddings, query_emb)
137
138          ranked = []
139          for idx, score in enumerate(sims):
140              chunk = self.chunks[idx]
141              if dept_filter and chunk.get("department_code") != dept_filter:
142                  continue
143              ranked.append({
144                  "chunk_id": chunk["chunk_id"],
145                  "score": float(score),
146                  "chunk": chunk
147              })
148
149          ranked.sort(key=lambda x: x["score"], reverse=True)
150          return ranked[:top_k]
```

How the libraries connect

np.load reads the saved embedding matrix and json.load supplies the corresponding passage records. np.dot multiplies each stored vector by the query vector. With unit-normalised document and query vectors, the dot product equals cosine similarity. Row ordering must stay aligned between the array and metadata; otherwise scores will be attached to the wrong sources.

Input and output: Question + matrix (1,039 x 384) + metadata -> ranked semantic candidates with scores and source-bearing chunks.

G.10 Combining Lexical and Semantic Rankings

Reciprocal rank fusion combines the two ranked lists by passage identifier. It uses rank positions rather than directly adding unlike raw score scales.

Source: backend\rag\retriever.py
Library imports: typing dictionaries and lists; enumerate; sorted

```
153  def reciprocal_rank_fusion(
154      dense_results: List[Dict[str, Any]],
155      sparse_results: List[Dict[str, Any]],
156      k: int = 60,
157      dense_weight: float = 1.0,
158      sparse_weight: float = 1.0,
159      top_k: int = 10,
160  ) -> List[Dict[str, Any]]:
161      """Fuses ranked candidate lists from dense semantic search and sparse lexical BM25 search."""
162      fused_scores: Dict[str, float] = {}
163      item_map: Dict[str, Dict[str, Any]] = {}
164      provenance: Dict[str, Dict[str, Any]] = {}
165
166      for rank, item in enumerate(dense_results, 1):
167          cid = item["chunk_id"]
168          item_map[cid] = item["chunk"]
169          fused_scores[cid] = fused_scores.get(cid, 0.0) + (dense_weight / (k + rank))
170          provenance.setdefault(cid, {})["dense_rank"] = rank
171          provenance[cid]["dense_score"] = item["score"]
172
173      for rank, item in enumerate(sparse_results, 1):
174          cid = item["chunk_id"]
175          item_map[cid] = item["chunk"]
176          fused_scores[cid] = fused_scores.get(cid, 0.0) + (sparse_weight / (k + rank))
177          provenance.setdefault(cid, {})["sparse_rank"] = rank
178          provenance[cid]["sparse_score"] = item["score"]
179
180      merged: List[Dict[str, Any]] = []
181      for cid, rrf_score in sorted(fused_scores.items(), key=lambda x: x[1], reverse=True):
182          merged.append({
183              "chunk_id": cid,
184              "rrf_score": round(rrf_score, 6),
185              "provenance": provenance[cid],
186              "chunk": item_map[cid]
187          })
188
189      return merged[:top_k]
```

How the libraries connect

Each passage accumulates a weighted contribution of 1/(k + rank) from every list in which it appears. The default k is 60. item_map preserves the passage while provenance records its original ranks and raw scores. A passage ranked first in both lists receives 2/61, approximately 0.032787, when both weights equal one.

Input and output: Dense and sparse hit lists -> fused candidates containing chunk_id, rrf_score, provenance and chunk.

G.11 Retrieval Orchestration and Output Mapping

This is the hybrid-search branch of HybridRetriever.retrieve. Earlier branches handle structured SQL facts and selected regulation queries.

Source: backend\rag\retriever.py
Library imports: BM25Retriever; DenseRetriever; reciprocal_rank_fusion; settings

```
394
395         # 2. Hybrid search: BM25 + Dense + RRF
396         sparse_hits = self.bm25_retriever.search(query, top_k=20, dept_filter=resolved_dept)
397         dense_hits = self.dense_retriever.search(query, top_k=20, dept_filter=resolved_dept)
398
399         fused = reciprocal_rank_fusion(
400             dense_results=dense_hits,
401             sparse_results=sparse_hits,
402             k=settings.RRF_K,
403             top_k=top_k * 2
404         )
405
406         formatted_chunks = []
407         for item in fused:
408             c = item["chunk"]
409             formatted_chunks.append({
410                 "chunk_id": item["chunk_id"],
411                 "text": c.get("content", ""),
412                 "content": c.get("content", ""),
413                 "course_code": c.get("course_code"),
414                 "course_name": c.get("course_title"),
415                 "course_title": c.get("course_title"),
416                 "source_document": c.get("source_document_id") or f"DOC-{c.get('department_code',
  |  'GEN')}",
417                 "source_document_id": c.get("source_document_id") or f"DOC-{c.get('department_code',
  |   'GEN')}",
418                 "page_number": c.get("source_page") or 1,
419                 "source_page": c.get("source_page") or 1,
420                 "retrieval_score": item["rrf_score"],
421                 "department_code": c.get("department_code"),
422                 "semester": c.get("semester_num"),
423                 "unit": c.get("unit_num"),
424             })
425
426         return formatted_chunks[:top_k]
```

How the libraries connect

Both retrievers receive the same resolved department and return up to 20 candidates. Fusion selects up to twice the requested top_k before the final mapping. The mapping standardises content, document identifiers, page numbers, department and unit fields so the answer generator does not need a different schema for each search method.

Input and output: Normalised question and entities -> standardised evidence dictionaries passed to generation and attribution.

G.12 Optional Cross Encoder Reranking

The core StudentChatbotPipeline can pass retrieved candidates to this reranker. The displayed block covers the transformer scoring branch, not the fallback heuristic below it.

Source: backend\rag\reranker.py
Library imports: from sentence_transformers import CrossEncoder (loaded in _init_model)

```
47          if not chunks:
48              return []
49
50          self._init_model()
51
52          # Extract text from chunk
53          def get_chunk_text(c: Dict[str, Any]) -> str:
54              if "chunk" in c and isinstance(c["chunk"], dict):
55                  return c["chunk"].get("content", "")
56              return c.get("text") or c.get("content", "")
57
58          # 1. Transformer Cross-Encoder Scoring
59          if self._model is not None:
60              pairs = [(query, get_chunk_text(c)) for c in chunks]
61              try:
62                  scores = self._model.predict(pairs)
63                  for idx, score in enumerate(scores):
64                      chunks[idx]["rerank_score"] = round(float(score), 4)
65                  chunks.sort(key=lambda x: x["rerank_score"], reverse=True)
66                  return chunks[:top_k]
67              except Exception as e:
68                  logger.warning("CrossEncoder inference execution failed (%s). Using heuristic.", e)
```

How the libraries connect

A cross-encoder reads a query and passage together. The code creates a pair for every candidate, predicts one relevance score per pair, attaches that score to the corresponding dictionary and sorts the candidates. This differs from dense retrieval, where document vectors are precomputed. It adds inference work after initial search. Its raw scores should not be treated as calibrated confidence.

Input and output: Question and candidate passages -> reordered top_k candidates carrying rerank_score. This stage is route-dependent.

G.13 Evidence Prompt Construction

The prompt builder joins selected evidence, source metadata and recent conversation text. The system directive is defined separately in the same module and supplied by the provider adapter.

Source: backend\rag\prompt_builder.py
Library imports: typing.List; typing.Dict; typing.Optional; string formatting

```
15  def build_rag_prompt(
16      query: str,
17      evidence_passages: List[str],
18      citations: List[Dict[str, Any]],
19      conversation_history: Optional[List[Dict[str, str]]] = None,
20      language: str = "en"
21  ) -> str:
22      """Construct prompt with evidence isolation and conversation context."""
23      history_text = ""
24      if conversation_history:
25          recent = conversation_history[-4:]
26          history_text = "Recent Conversation History:\n" + "\n".join(
27              [f"{m['role'].capitalize()}: {m['content']}" for m in recent]
28          ) + "\n\n"
29
30      evidence_text = ""
31      for idx, (passage, cit) in enumerate(zip(evidence_passages, citations), 1):
32          doc_id = cit.get("doc_id", "DOC")
33          page = cit.get("page", 1)
34          dept = cit.get("department", "Aditya University")
35          evidence_text += (
36              f'<evidence index="{idx}" doc_id="{doc_id}" page="{page}" department="{dept}">\n'
37              f"{passage.strip()}\n"
38              f"</evidence>\n\n"
39          )
40
41      prompt = (
42          f"{history_text}"
43          f"Verified Official University Evidence:\n"
44          f"{evidence_text if evidence_text else '<evidence>No matching official documents
 |  found.</evidence>\n'}\n"
45          f"Student Inquiry: {query}\n\n"
46          f"Provide a clear, helpful, academically grounded answer:"
47      )
48
49      return prompt
```

How the libraries connect

Each evidence block carries a document identifier, source page and department. zip pairs passages with citations, so mismatched list lengths would silently omit extra items. The history slice keeps at most four messages supplied to this function. Although language is accepted as an argument, this function does not currently branch on it. Evidence tagging alone is not a complete security boundary.

Input and output: Question, passage strings, matching citation metadata and optional history -> one prompt string.

G.14 Generation Adapter Selection

The adapter interface separates answer generation from retrieval. The factory below is the implemented provider-selection logic in this project.

Source: backend\rag\llm_adapter.py
Library imports: backend.config.settings; backend.rag.prompt_builder.SYSTEM_DIRECTIVE

```
 16  class BaseLLMAdapter:
 17      """Abstract base class for LLM backends."""
 18
 19      def generate_response(self, prompt: str, system_directive: str = SYSTEM_DIRECTIVE) -> str:
 20          raise NotImplementedError

203  def get_llm_adapter() -> BaseLLMAdapter:
204      """Factory selecting configured LLM provider with graceful fallbacks."""
205      provider = settings.LLM_PROVIDER.lower()
206
207      if provider == "gemini" and settings.GEMINI_API_KEY:
208          return GeminiAdapter()
209      elif provider == "openai" and settings.OPENAI_API_KEY:
210          return OpenAIAdapter()
211      else:
212          return DeterministicOfflineAdapter()
```

How the libraries connect

AnswerGenerator calls generate_response through a shared interface. The factory selects Gemini only when its provider setting and key are present, OpenAI only when its corresponding setting and key are present, and the deterministic offline implementation otherwise. The SDK imports reside inside the provider classes; installing the base backend requirements does not guarantee those SDKs are available.

Input and output: Server configuration -> a generation adapter; prompt and system directive -> answer text.

G.15 NLP and ML Data Flow

The training and inference flows below connect the source excerpts in this report. They describe the implementation and are not a newly executed model evaluation.

Stage	NLP or ML data transformation
Corpus preparation	Curriculum PDF pages become text and source metadata.
Query preprocessing	Regular expressions, protected tokens and optional lemmatisation produce normalised text.
Feature learning	Word and character TF-IDF vectors are concatenated by FeatureUnion.
Intent training	A soft-voting ensemble learns the labelled question categories.
Model inference	The saved pipeline maps a question to class probabilities and an intent.
Evidence retrieval	BM25 and sentence embeddings score relevant passages; rank fusion merges candidates.
Optional reranking	A cross-encoder scores query-passage pairs and orders candidates.
Prompt and generation	Selected passages and source metadata form the evidence prompt used by the generation adapter.

Table G.3: Data flow through the NLP and ML components

Integration Checks

Keep preprocessing identical during training and prediction. Preserve the class order used by probability columns. Verify that vector dimensions and metadata row order match before dense search, and rebuild embeddings when changing the encoder.

Evaluate classifier accuracy, retrieval relevance and answer correctness separately. BM25 scores, fused ranks, cross-encoder scores and classifier probabilities have different meanings; none alone establishes factual correctness.

G.16 Documentation and Code Provenance

All code excerpts are drawn from the project source. Library integration descriptions distinguish the trained intent model, pretrained semantic encoders, retrieval algorithms and optional language generation.

[L1] scikit-learn. FeatureUnion combines parallel feature transformations.
FeatureUnion API documentation

[L2] Sentence Transformers. Text encoding, normalisation and output configuration.
SentenceTransformer API documentation

[L5] PyMuPDF. Extracting text from document pages.
PyMuPDF text extraction documentation

Source Boundaries

The listings retain the implemented logic and its documented limitations. They are excerpts for academic explanation, not standalone replacements for the source modules. Printed line numbers refer to the source files; a vertical bar indicates a wrapped source line.

Appendix A supplies the feature and ensemble builders, Appendix B supplies preprocessing, and Appendix C supplies rule-based entity extraction. Appendix G expands the NLP and ML pipeline with library imports, source excerpts, input/output descriptions and explanations.

No new training run or benchmark is claimed. No application source, trained model or academic dataset was changed for this report revision.